Keeping a Step Checker Off the Critical Path: Measured Savings from Asynchronous Verification, KV-Cache Masking and Loop Cropping

Aditya Bankoti
University of Washington
Technical report, October 2026

Abstract. A language model that reasons in written steps makes mistakes in individual steps. If each step is also written as a logic formula, an automated prover can check it against the problem's premises. This report measures the cost of running such a checker alongside a small language model, and three ways of lowering it, on ProverQA-Hard with Qwen3 models of 8B, 14B and 30B parameters and simulated checker delays of 1, 3 and 8 seconds, then on five more models from four families at a smaller sample. All comparisons are paired on the same problems. Letting the model keep writing while the checker works is faster than pausing for each verdict on every model at every delay ($p < 0.001$), by 12 to 67%. When a step is refuted, deleting only it and the steps built on it (masking) beats deleting everything after it (truncation) on the 30B at every delay and on the 14B at 8 s ($p \leq 0.04$), by 4 to 10%; on the Qwen3-8B at 8 s it instead costs accuracy (0 wins, 8 losses, $p = 0.008$), a failure that recurs on none of the other seven models. Deleting steps that repeat an earlier step removes 19% of generated text and 16% of time on the 14B ($p \leq 0.01$) without changing an answer. The checked pipeline beats plain chain-of-thought on every model, but which half does the work depends on the family: on Qwen3 the step format alone carries the gain (8 to 20 points) and the checker adds nothing; on Llama, Phi and Mistral the format alone does nothing and the checker carries it (pooled 17 wins to 3 losses, $p = 0.003$).

1 Introduction

Language models of 8 to 30 billion parameters can solve multi-step logic problems, but a wrong step early in the chain corrupts everything after it. If each step is written both in English and as a formula in formal logic, a program can check the formula against the problem's premises and catch the mistake when it is made. This report is about such a pipeline: a small language model writes numbered steps, each with a formula and a list of the earlier steps it relies on; the prover Z3 (Section 2.3) checks each formula; and a step that contradicts the premises is removed together with every later step that relied on it.

The step format made the models more accurate; the checker on top of it did not (Section 6.1). What we measured is the checker's cost, which depends on three choices: whether the model waits for each verdict; whether a refuted step takes everything after it with it, or only the steps that depended on it; and whether obviously wasted text, such as a step that repeats an earlier one, is cut. Each choice was tested as a paired comparison on the same problems. We report only differences that are statistically significant; a companion report covers every experiment, including those that found nothing.

1. **The checked pipeline beats plain chain-of-thought on every model, but the gain comes from a different half depending on the family.** On the five Qwen3 models the step format alone raises accuracy by 8 to 20 points and the checker adds nothing; on Llama 3.1, Phi-4 and Mistral the format alone does nothing and the checker carries the gain (pooled 17/3, $p = 0.003$) (Section 6.1).
2. **Letting the model keep writing while the checker works ("asynchronous") beats waiting for each verdict ("blocking")** on every model at every delay, saving 12 to 67% of the time per problem with the same answers (Section 6.2).
3. **On the two larger models, deleting only a refuted step and the steps built on it ("masking") beats deleting everything after it ("truncation")** by 4 to 10%: on the 30B at every delay, on the 14B when the checker is slow (Section 6.3).

4. **On the 8B with a slow checker, masking costs accuracy:** 36.7% correct against 63.3% under truncation ($p = 0.008$). The loss recurs on none of the other seven models, including a 4B and an 8B of another family, so it is one model's failure rather than a size threshold (Section 6.3).
5. **Deleting steps that repeat an earlier step ("loop cropping") removes 19% of the generated text and 16% of the time on the 14B,** with identical answers (Section 6.4).

Terms.

- **First-order logic (FOL) formula.** A statement built from predicates such as `is_open_minded(Raphael)` with ¬ (not), ∧ (and), ∨ (or), ⟶ (implies), ⊕ (exactly one of), ∀ (for all) and ∃ (there exists).
- **Z3.** An automated prover: given a set of formulas, it decides whether they can all be true at once (Section 2.3).
- **Verdict.** What the checker says about a step's formula: **entailed** (the premises force it true), **refuted** (they force it false) or **unknown**.
- **Checker delay L.** The time from a step being written to its verdict arriving; simulated as 1, 3 or 8 seconds (Section 2.3).
- **Blocking, asynchronous.** Whether the model waits for each verdict before writing the next step, or keeps writing (Section 4.1).
- **Truncation, masking.** The two ways of removing a refuted step and its dependents: delete everything after it, or delete only those steps (Section 4.2).

2 Background

2.1 The task

Each problem in ProverQA-Hard [1] is a list of premises about a person followed by a statement, and the answer is whether the statement is true, false or uncertain given the premises. The benchmark was generated by a symbolic prover, so every premise comes with its logic formula and the labels are correct; Z3 reproduces the published label on 498 of the 500 problems. Figure 1 shows one, with what the model wrote.

Premises (problem 342). Each comes with its logic formula; two are shown.

P1 Everyone who values critique either embraces debate or is open-minded (or both).

P2 Anyone who considers alternatives and questions assumptions can challenge the status quo.

P3 Raphael has extensive knowledge.

P4 Raphael does not write influential books.

P5 Raphael is either a respected authority or has unconventional views, but not both.

```
respected_authority(Raphael)
⊕ has_unconventional_views(Raphael)
```

P6 Anyone who has extensive knowledge and speaks authoritatively gains recognition.

P7 Raphael is either a respected authority or a doubted expert, but not both.

P8 Raphael does not embrace debate.

P9 Raphael does not gain recognition.

P10 If Raphael challenges the status quo, then he either writes influential books or speaks authoritatively (or both).

P11 If someone is open-minded and has unconventional views, then they challenge the status quo.

```
∀x ((is_open_minded(x) ∧ has_unconventional_views(x))
→ challenges_status_quo(x))
```

P12 Raphael values critique.

Statement. Raphael is a doubted expert. **Gold label:** False.

What the 14B model wrote, with the checker's verdicts (asynchronous run, 8-second checker, masking):

Step	uses	Claim, and the formula the model wrote for it	Verdict	What happened
S1	P12, P1	Raphael either embraces debate or is open-minded. `embraces_debate(Raphael) ∨ is_open_minded(Raphael)`	not a fact	kept, unchecked
S2	S1, P8	Raphael is open-minded. `is_open_minded(Raphael)`	entailed	kept
S3	S2, P5	Raphael has unconventional views. `has_unconventional_views(Raphael)`	**refuted**	removed
S4	S2, **S3**, P11	Raphael challenges the status quo.	(pending)	removed: built on S3
S5	**S4**, P10	Raphael either writes influential books or speaks authoritatively.	(pending)	removed: built on S4
S6	**S5**, P4	Raphael speaks authoritatively.	(pending)	removed: built on S5
S7	**S6**, P3, P6	Raphael gains recognition.	(pending)	removed: built on S6

S3 is wrong: P3, P9 and P6 give that Raphael does not speak authoritatively; with P4 and P10, he does not challenge the status quo; with P11 and S2, he cannot have unconventional views. The verdict arrived after the model had written S4 to S7, each of which uses the step before it, so all five were removed and a note was appended. The model continued from S2 and answered False, correctly.

Figure 1: A worked example. The third step contradicts the premises, and the four steps that declared a dependency on it are removed with it.

2.2 Reasoning in checked steps

The model is asked to reason in numbered steps of the form

```
Step k [uses P2, S1]: ⟨claim in English⟩
FOL: ⟨the same claim as a formula⟩
```

ending with `Answer: A/B/C`. The `[uses]` list names the premises (`P`) and earlier steps (`S`) the step is derived from, so the steps form a dependency graph; in Figure 1 it is the chain S3 → S4 → S5 → S6 → S7. The model writes the list itself, and we take it at face value.

2.3 What Z3 does, and why its delay is the variable

Z3 is an SMT solver: given a set of formulas, it decides whether they can all be true at once. For a step's formula φ and the premises Γ we ask it two questions. If Γ together with $\neg\varphi$ cannot all be true, the premises **entail** φ. If Γ together with φ cannot all be true, they **refute** φ. Otherwise the verdict is **unknown**. Only refuted steps are removed. A formula that is not a plain fact (S1 in Figure 1, an "or") is kept unchecked. Z3 sees only the formula, never the English, so a step with a correct claim and a wrong formula is judged on the formula; this is one reason the checker does not improve accuracy (Section 6.1).

On this benchmark Z3 answers in milliseconds, so a pipeline deployed as built would lose almost nothing by waiting for it. A realistic pipeline is slower in front of Z3: the English step has to be translated into logic by a separate model call, or the check is done by a proof assistant such as Lean, or the checker shares the GPU with generation. We therefore treat the checker's delay (latency) L as the variable and simulate a checker that returns each verdict L seconds after the step was written, for L = 1, 3 and 8 s. The 8-second setting stands for a proof assistant or a loaded GPU; 1 second is close to a fast translation model.

3 Prior work

Three findings frame this report. Checking reasoning steps with a formal prover catches real errors, whether after the chain is complete [2, 4] or during generation by regenerating from the last verified step [3] benchmarks with gold formulas make it possible [1], and dependency graphs over steps have been used to trace how a wrong step corrupts later ones [5]. Deleting reasoning tokens from the KV cache during generation saves memory and can improve accuracy when the deleted tokens are chosen by a budget or a learned signal [6, 7, 8]. And deleting a span from the cache does not erase its influence, because later tokens were computed while it was visible; exact erasure requires recomputing everything after the span [9, 10, 11]. Retracting a conclusion together with everything derived from it is an old idea in artificial intelligence and databases [12, 13]. Not previously measured: a prover's verdict as the trigger, the model's own dependency list as the scope, a live KV cache as the target, and checker delay as the variable.

4 The pipeline

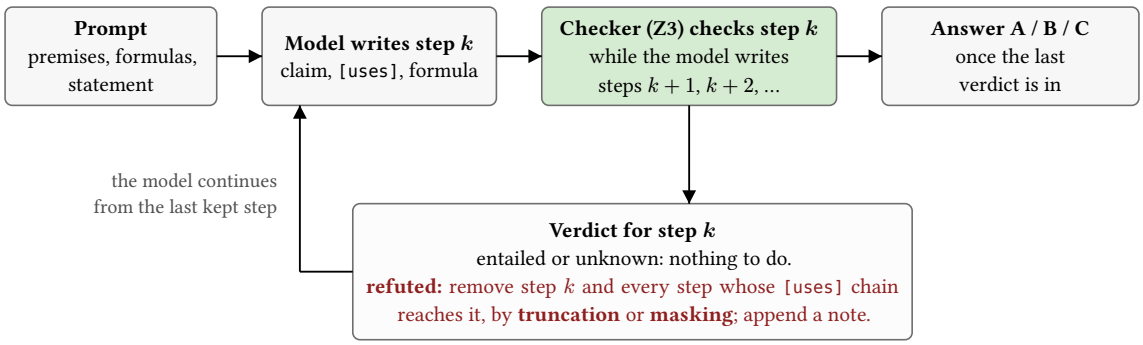


Figure 2: The pipeline. The checker runs beside generation; a refuted verdict removes the step and its dependents and the model continues from the last kept step.

4.1 Blocking or asynchronous

Under blocking ("sync") the model waits for each verdict, so every step costs L seconds of idle time. Under asynchronous checking it keeps writing; a verdict is applied when it arrives, usually one or two steps later, and only the final answer waits for outstanding verdicts (Figure 3). The cost of not waiting is that a refuted step may already have steps built on it, as in Figure 1.

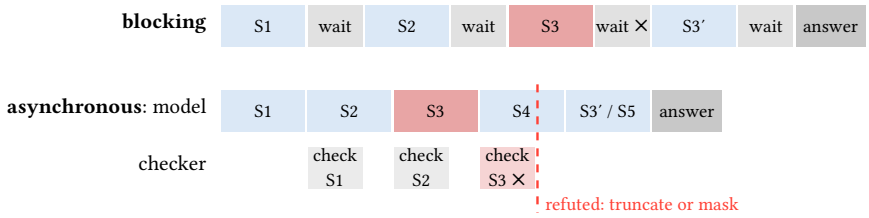


Figure 3: Blocking pays the checker's delay after every step. Asynchronous checking pays only when a step is refuted, and then only to rewrite the steps that have to go.

4.2 Truncation or masking

When step S_k is refuted, the steps to remove are S_k and every later step whose `[uses]` chain leads back to it. **Truncation** deletes the cache from the start of S_k onward, and the model rewrites everything after it, including later steps that never used S_k. **Masking** deletes only the entries of S_k and its dependents, with llama.cpp's `kv_cache_seq_rm`, and the model continues after the last kept step (Figure 4).[1] Masking saves the rewriting. Its risk is that the kept steps' cache entries were computed while the deleted step was still visible, so they can carry its content [9, 10, 11] Section 6.1 measures whether that matters. Under either policy, three refutations in a row make the harness back up to the last accepted step.

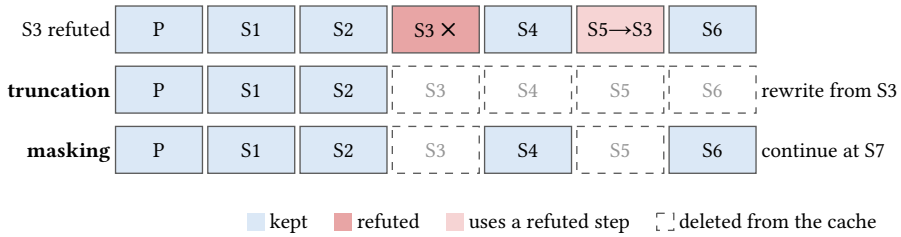


Figure 4: The two policies on the same cache. S5 declared `[uses S3]`, so it goes too; S4 and S6 are independent and survive masking.

4.3 Loop cropping

In every condition, including runs with no checker, the model sometimes repeats one formula step after step until it hits the step limit. The detector compares each new step's formula with the earlier live steps'; a repeat is deleted and replaced by a note, and after three repeats the model is made to answer (Figure 5).

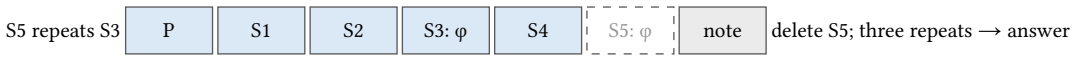


Figure 5: Loop cropping: a step whose formula repeats an earlier step's is deleted and noted.

4.4 Simulating the delay

The delay is simulated on a token clock. The 8B model wrote about 14 tokens per second on the reference machine (72.4 ms per token), so a verdict due L seconds after a step arrives once the model has written another $14L$ tokens. Time per problem is then computed from the transcript as

$$T = 8.6\ \text{s} + 0.0724\ \text{s} \times \text{generated tokens} + \text{time waiting for verdicts},$$

where 8.6 s is the measured fixed cost per problem (prompt, Z3). Under blocking the wait is L per check; under asynchronous checking it is whatever is still outstanding at the answer. We call T **modeled time**. Using one formula for every model keeps the policies comparable across models and machines; GPU wall time is also reported and moves the same way.

[1] Masking sets the attention weight of the deleted positions to zero for every later token. The remaining entries keep their positions; llama.cpp tolerates a gap in the middle of the cache but not at its end, so a removal that reaches the end is done as a truncation.

5 Experimental setup

Benchmark. ProverQA-Hard [1], the 500-problem hard split: 6 to 9 reasoning steps per problem, distractor premises, three labels. Always answering Uncertain scores about 40%. Problems are sampled with a fixed seed: 30 for the Qwen3-8B and the five later models, 100 for the Qwen3-14B and 30B. The loop-cropping run uses a 30-problem sample of which 28 are among the 100; its comparisons use those 28.

Models. Qwen3-8B, Qwen3-14B and Qwen3-30B-A3B (a mixture of experts with 3B parameters active per token), 4-bit quantized, run with llama.cpp in non-thinking mode, greedy decoding, temperature 0.7 after a rejection. The 8B ran on an Apple M4 laptop (24 GB); everything else on a Colab L4 GPU (24 GB), about 18 GPU-hours in all. A later sweep added Qwen3-4B, Llama 3.1 8B, Phi-4 (14B), Mistral Small 24B and the dense Qwen3-32B, same quantization and harness with each model's own chat template, on a Colab A100 for 3.3 GPU-hours (Figure 7, Figure 9).

Conditions. `sync`: blocking. `trunc`, `mask`: asynchronous with the named policy, at $L \in \{1, 3, 8\}$ s. The loop-cropping run repeats `trunc` and `mask` on the 14B at $L = 8$ s with the detector on.[2]

Metrics. Accuracy; generated tokens per problem; modeled time (Section 4.4); L4 wall time.

Statistics. Every comparison is between two runs on the same problems. For each problem we record which run was faster (for accuracy, which run alone was right). If the runs were equivalent each problem would be a coin flip, so a split such as 25 faster against 11 slower is unlikely by chance: the probability of a split at least that uneven, p, is 0.03. We report the split as wins/losses with p from the two-sided exact sign test (McNemar's test for accuracy) and call $p < 0.05$ significant. With 30 problems the smallest significant split is 6 to 0, so the 30-problem runs can show the speed effects, which are large, but could miss an accuracy change of a few problems.

6 Results

The figures below show the comparisons that were significant. Every comparison not mentioned was not: the checked policies' accuracies differ by at most a few problems on every model at every delay ($p \geq 0.06$), and the checked pipeline is not significantly different from the same step format with no checker ($p \geq 0.11$). The full tables are in the companion report.

6.1 Accuracy: the format or the checker helps, by family; masking hurts one model

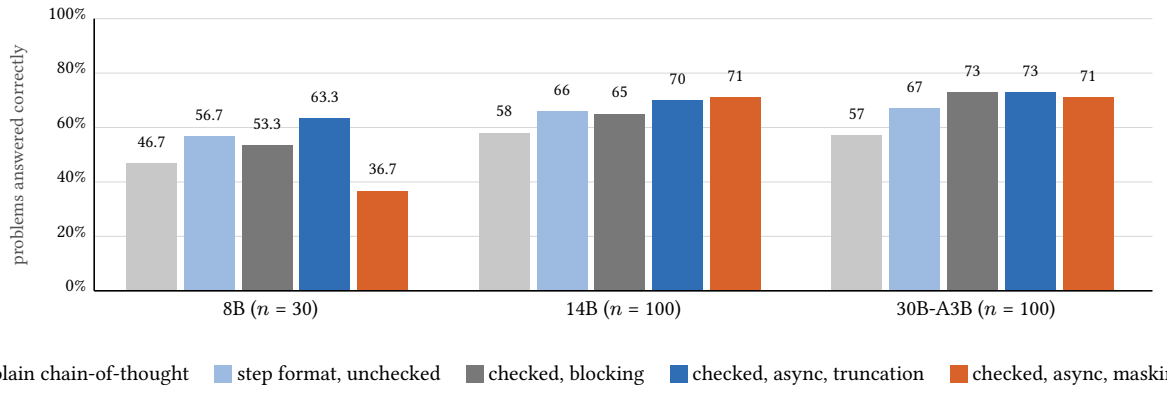


Figure 6: Accuracy; the checked conditions use the 8-second checker. Two differences are significant: the step format over plain chain-of-thought (pooled across the three models, 55 wins / 34 losses, $p = 0.03$), and masking against truncation on the 8B (0 wins / 8 losses, $p = 0.008$). At 1 and 3 s the checked bars are within 3 points of these, except 8B masking (60.0 and 53.3, not significant).

[2] That run also let the final answer wait only for the verdicts of the steps it depends on. On problems where nothing was cropped this changed nothing: transcripts and times are identical. On the cropped problems it saved about one second of waiting, which is included in the reported total.

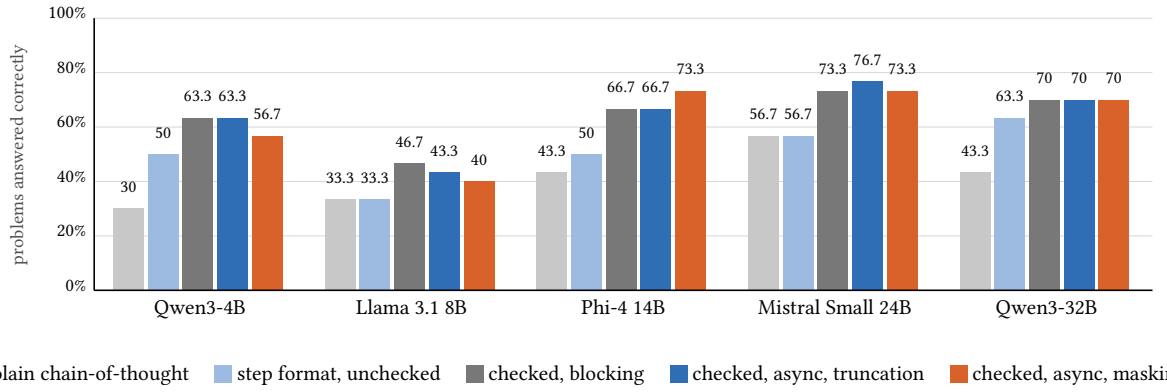


Figure 7: Accuracy on the five later models, 30 problems each, checked conditions at 8 s. On the two Qwen sizes the step format carries the gain (pooled 19/7 over plain chain-of-thought, $p = 0.03$) and the checker adds nothing significant. On the three other families the format alone does nothing (18/16) and the checker carries the gain instead (blocking checker over unchecked format, pooled 17/3, $p = 0.003$; async policies at 8 s, 22/8, $p = 0.02$).

Two things in Figure 6 are significant. First, the step format alone, with no checker, raises accuracy over ProverQA's own chain-of-thought prompt by 10, 8 and 10 points on the three models (6/3, 25/17 and 24/14 wins/losses; pooled 55/34, $p = 0.03$; on the 30B with the step cap removed, 27/13, $p = 0.04$). The prompt also shows the formula of every premise, which may account for part of this. Adding the checker to the format changes nothing: the checked policies get the same problems right as the unchecked format and as each other. Second, the one exception among the checked policies: masking on the 8B with the 8-second checker loses 8 problems to truncation and wins none. That model does not recover from a span cut out of the middle of its context; the seven others do (Section 6.3).

The five later models (Figure 7) split the first finding by family. The two new Qwen3 sizes repeat the Qwen pattern: the format alone gains 20 points on both (pooled 19/7, $p = 0.03$) and the checker adds nothing significant. Llama 3.1 8B, Phi-4 and Mistral Small 24B do the opposite: the format alone gains nothing (18/16 pooled), and the checker on top of it gains 14 to 24 points (blocking checker over unchecked format 6/2, 6/1 and 5/0; pooled 17/3, $p = 0.003$; the asynchronous policies keep most of it, 22/8 at 8 s). Where the checker helps, it helps on problems where it refuted something: of the 22 problems those three models get right only with the checker, 17 had at least one refuted step. Thirty problems per model is too few to rank the five, so the claim is the pooled one: every family ends up ahead of plain chain-of-thought (masking at 8 s over plain, pooled across the five, 49/17, $p < 0.001$), and a model that already follows the step format well has little left for the checker to fix. Everything below is cost at equal accuracy.

6.2 Asynchronous checking against blocking

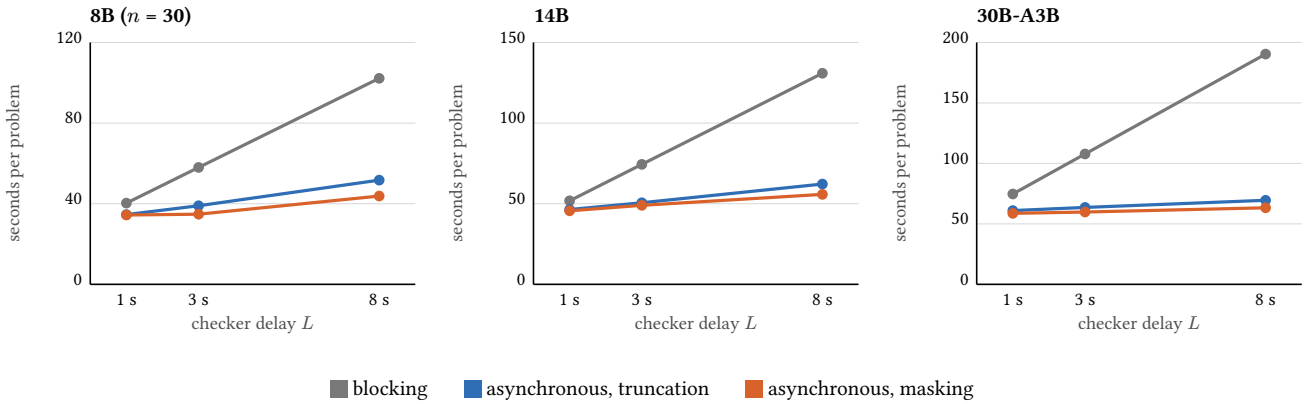


Figure 8: Modeled seconds per problem against the checker's delay. Every asynchronous point is below the blocking point above it at $p < 0.001$.

Not waiting is faster on every model at every delay: faster on 27 to 29 of 30 problems on the 8B and on 77 to 99 of 100 on the 14B and 30B. The saving grows with the delay, from 12 to 21% at 1 s to 49 to 67% at 8 s, because blocking pays the delay after every step while asynchronous checking pays only to rewrite refuted steps.

6.3 Masking against truncation

	time saved			accuracy change		
	1 s	3 s	8 s	1 s	3 s	8 s
Qwen3-8B n = 30	0.6% p=1	10.8% p=0.18	15.2% p=0.15	−3.3 p=1	−10.0 p=0.25	**−26.7** p=0.008
Llama 3.1 8B n = 30	1.9% p=1	4.1% p=0.23	11.8% p=0.3	0.0 p=1	+3.3 p=1	−3.3 p=1
Qwen3-14B n = 100	1.8% p=0.18	3.1% p=0.23	**10.4%** p=0.003	0.0 p=1	+1.0 p=1	+1.0 p=1
Phi-4 14B n = 30	4.6% p=0.23	4.3% p=0.39	13.3% p=0.1	+10.0 p=0.25	+10.0 p=0.38	+6.7 p=0.73
Mistral Small 24B n = 30	5.8% p=0.45	7.6% p=0.51	8.1% p=1	0.0 p=1	−3.3 p=1	−3.3 p=1
Qwen3-30B-A3B n = 100	**3.8%** p=0.04	**5.9%** p=0.04	**8.9%** p=0.03	+2.0 p=0.63	−1.0 p=1	−2.0 p=0.75
Qwen3-32B n = 30	1.6% p=0.25	3.7% p=0.06	0.9% p=1	−3.3 p=1	−3.3 p=1	0.0 p=1
Qwen3-4B n = 30	−1.5% p=0.34	1.8% p=0.3	19.7% p=0.45	+3.3 p=1	+3.3 p=1	−6.7 p=0.63

columns: checker delay colour depth = significance (p in each cell), scaled to each block's strongest cell **bold** = $p < 0.05$ rows by parameter count

Figure 9: Masking against truncation, per model and checker delay: time saved (left) and the change in accuracy (right). Time saved grows with the delay on every model but the Qwen3-32B, which has little to keep: a mask salvages 9 tokens per problem there against 30 to 89 on the other models. Accuracy is unchanged on every model but the Qwen3-8B, where the loss grows with the delay. The savings are significant on the Qwen3-30B at every delay and on the 14B at 8 s (n = 100); the 30-problem runs cannot confirm savings of this size.

With asynchronous checking in place, masking saves a further 4 to 10% of time on the Qwen3-14B and 30B, and the saving grows with the delay because a later verdict leaves more independent work to keep (Figure 9). On the Qwen3-8B, and on the 14B at 1 and 3 s, the two policies are not distinguishable on time. The accuracy cost of masking is −27 points on the Qwen3-8B at 8 s and within ±7 points everywhere else, including the smaller Qwen3-4B (−6.7, 1 win, 3 losses) and an 8B of another family (Llama 3.1, −3.3, 2 wins, 3 losses). So the failure is not a matter of parameter count. For the 30-problem rows, that sample is enough to rule out a collapse like the Qwen3-8B's but not to confirm a 5 to 13% time saving, so those cells are not bold.

6.4 Loop cropping

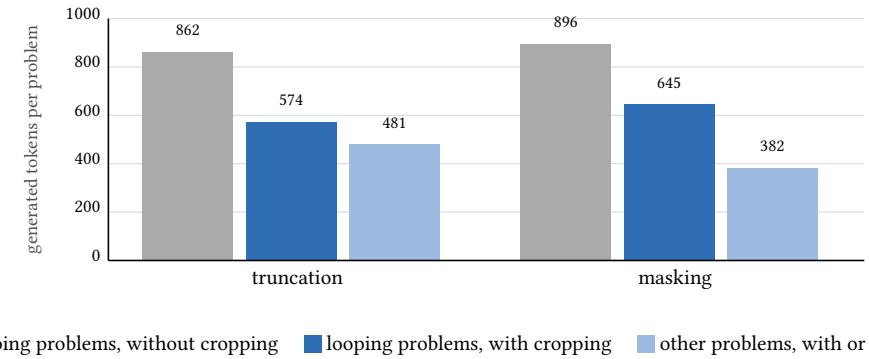


Figure 10: Generated tokens per problem on the 14B at L = 8 s, split by whether the detector fired (12 of 28 problems under truncation, 13 under masking). On the other problems the transcripts are identical with and without cropping.

Cropping repeated steps saves 16% of modeled time and 19% of generated tokens under both policies (truncation 62.1 → 52.1 s, masking 60.3 → 51.5 s), faster on 11 of 12 and 12 of 13 differing problems (p = 0.01 and 0.003). Answers did not change (1 win, 1 loss per policy), so the cropped tokens were waste.

7 Where this applies

The savings apply wherever a verifier is slower than generation and can run beside it: an autoformalizer that turns each step into logic before an SMT check, a proof assistant such as Lean, unit tests run on each generated function, a tool or API call whose result confirms or refutes a step, or a second model acting as a critic. In each case asynchronous checking lets generation continue and charges only for refuted steps. Dependency-scoped removal keeps the work that did not depend on the refuted step, which pays once the model is 14B or larger and the verifier is slow; below that, truncation. Loop cropping applies to any step-structured output. The pipeline needs two things from the task: steps that can be checked one at a time, and a dependency list the model can be trusted to write. We tested only the Z3 case; the others share its structure but were not measured here.

8 Limitations

- The delay is simulated. A real slow checker, such as a translation model sharing the GPU, would also slow generation itself.
- One benchmark, with gold premise formulas: the checker verifies only the formulas the model writes for its steps.
- The Qwen3-8B, the five later models and the loop-cropping run use 30 problems, enough for the speed effects and for a collapse but not for a small accuracy change or a 5 to 15% masking saving. The 14B and 30B main results use 100.
- The dependency lists are the model's own and were not verified independently here.
- All models are 4-bit quantized, with one decoding setting and one seed.

9 Conclusion

Three changes make a prover-checked reasoning pipeline cheaper at the same accuracy: asynchronous checking (12 to 67% faster than blocking, on every model and delay), masking a refuted step's dependents instead of truncating (4 to 10% faster on the 14B and 30B with a slow checker, harmful on the Qwen3-8B only among eight models), and cropping repeated steps (16% faster on the 14B). The checked pipeline beat plain chain-of-thought on every model; on Qwen3 the step format did the work, on three other families the checker did. Next: replace the simulated delay with a real slow checker and repeat the comparison on at least 100 problems; make masking safe for small models by letting each step attend only to the prompt and the steps it declares, which removes the leakage described in Section 4.2 by construction; and, since the checker's accuracy gain on the non-Qwen families rests on 30 problems per model, repeat that comparison on a few hundred.

Artifacts. Harness: `run.py`, `logic.py`, `analyze.py`, `analyze_eff5.py`; cloud driver: `cloud/colab.sh`, `cloud/vm.sh`; raw runs: `results/v2/` (8B), `results_cloud/Qwen3-14B/`, `results_cloud/Qwen3-30B-A3B/`, `results_cloud/Qwen3-14B_eff/`, `results_cloud/{Qwen3-4B,Llama-3.1-8B,Phi-4,Mistral-Small-24B,Qwen3-32B}/` (`analyze_r6.py`); every experiment including the null results and full tables: `results/RESULTS.md`, `results/paper_full.pdf`.

10 References

[1] C. Qi, R. Ma, B. Li, H. Du, B. Hui, J. Wu, Y. Laili and C. He. *Large Language Models Meet Symbolic Provers for Logical Reasoning Evaluation* (ProverQA). ICLR 2025. arXiv:2502.06563.

[2] Y. Feng, N. Weir, K. Bostrom, S. Bayless, D. Cassel, S. Chaudhary, B. Kiesl-Reiter and H. Rangwala. *VeriCoT: Neuro-symbolic Chain-of-Thought Validation via Logical Consistency Checks.* arXiv:2511.04662, 2025.

[3] J. Hu, S. Yang, W. Liu and D. Wang. *LogicTrack: Auditing Reasoning Trajectories of Large Language Models with Formal Logic Solvers.* arXiv:2609.21492, 2026.

[4] C. Liu et al. *Safe: Enhancing Mathematical Reasoning in Large Language Models via Retrospective Step-aware Formal Verification.* ACL 2025. arXiv:2506.04592.

[5] S. Mukherjee, A. Chinta, T. Kim, T. A. Sharma and D. Hakkani-Tür. *Premise-Augmented Reasoning Chains Improve Error Identification in Math Reasoning with LLMs* (PARC). ICML 2025. arXiv:2502.02362.

[6] T. Jin. *Self-Orchestrating Language Models: Leveraging Semantic Dependence for Efficient Inference* (chapter 4, "Thinking in Place"). arXiv:2609.14850, 2026.

[7] D. Choi et al. *Think Clearly: Improving Reasoning via Redundant Token Pruning*. arXiv:2507.08806, 2025.
[8] B. Feng et al. *StepKV: Step-Aware KV Cache Compression for LLM Agents*. arXiv:2609.22158, 2026.
[9] C. Zhou et al. *Positions Are Not Facts: The Mismatch Between KV Caches and Memory*. arXiv:2609.33759, 2026.
[10] G. Zhang and H. Yang. *Aborted but Not Forgotten: KV-Cache Retention Breaks Rollback Consistency in Language Agents*. arXiv:2608.15939, 2026.
[11] M. Li et al. *KVEraser: Learning to Steer KV Cache for Efficient Localized Context Erasing*. ICML 2026 Workshop on the Impact of Memorization on Trustworthy Foundation Models. arXiv:2606.17034.
[12] J. Doyle. *A Truth Maintenance System*. Artificial Intelligence 12(3), 1979. doi:10.1016/0004-3702(79)90008-0.
[13] A. Gupta, I. S. Mumick and V. S. Subrahmanian. *Maintaining Views Incrementally*. SIGMOD 1993. doi:10.1145/170035.170066.